

url4 topology — endpoint, node, discovery, addressing, transport

Sharp definitions before the Engine grows further

STATUS	proposed — owner review; one decision left open in §5; reconciled 2026-09-15 with Kevin's Part A §1.4 refresh, PR #19 commit 2a939bff (draft, unmerged) — see §12 and Appendix D
WORK ITEM	OME-1110
DATE	2026-09-04
OWNER	Sergey Bershadsky (execution and telemetry architecture)
GRAMMAR OWNER	Kevin McDonough (URL4 spec, Parts A/B)
SPEC POINTERS	URL4.ai Specification v0.5 DRAFT — Part A §1.4 at PR #19 (2a939bff), Parts A/B on main, Parts C–I drafts on url4-refactor

Read §0 first. Everything else is the reasoning behind one line of that table.
Open questions and spec deltas live in the companion, open-work PDF.

0. The eight answers

This document fixes the words for the parts of url4. It is short on purpose. Each section gives one definition, says where the spec agrees, and stops.

Updated 2026-09-15 against Kevin's Part A §1.4 (PR #19, [2a939bff](#), draft). He took most of what we proposed. Where we differ, his text wins. §12 lists the changes; Appendix D maps every term.

#	QUESTION	ANSWER
1	What is an endpoint? What is a node?	An endpoint is an interface on a node. It has an endpoint path (<code>/claude</code>). It takes <code>(context)!intent</code> , returns a result, and can evaluate a full expression. It keeps no state (§3). A node is the origin that serves endpoints. <code>/</code> is its default.
2	Who owns what?	The node owns discovery, credentials and outbound traffic (§4, §11). An endpoint owns only its function. Every requestor that runs an evaluator is itself a node (§4, §13).
3	Swarm? Composition?	Not protocol words. The expression is the composition. The nodes it touches are its request tree. "Swarm" is a deployment word for one operator's nodes.
4	<code>.well-known</code> or OPTIONS?	Three ways: the node document (Part G §27.1), the Capabilities response header (also §27.1), and OPTIONS (ours). §5 shows all three. The owner picks.
5	<code>/name</code> vs <code>url4://name</code> ?	<code>/name</code> is an endpoint on the node doing the evaluating. <code>url4://name</code> is node <code>name</code> 's default endpoint <code>/</code> . Part B §5.4 says this already.
6	Where does an ensemble run?	Wherever an evaluator runs: the SDK on a laptop, or a node. A local node may mount remote endpoints under local names (proxy mounts).
7	Can I test a node first?	Fetch its capabilities document, or send one OPTIONS per endpoint (§5). A dry run — evaluate, do not execute, return the envelope with no result — is now a spec word (Part A §1.4.3). Only its shape is open (§10).
8	Streaming fallback?	One GET asks for the richest mode. The endpoint answers with the best it has: WebSocket, then SSE, then sync (§6). Sync is the only MUST. The spec calls this the response ladder and adds <code>websocket</code> as a fourth delivery value (Part A §1.4.4). Part C is not updated yet.
9	Text in, text out?	A habit, not a rule. Every edge carries a typed payload (text, image, audio, video, embeddings) named by its media type. Text is the default. No grammar change (§8).

What changes against today

- The Engine becomes a url4 **node**. Today no url4 endpoint is reachable over HTTP. The SDK's endpoint-to-endpoint code is unused.
- Every endpoint speaks the same GET. It picks the richest delivery it has: **WebSocket** → **SSE** → **sync**, in one round trip. Sync is the only MUST.
- We keep the spec's two words. A **node** serves **endpoints** (`/claude`, `/codex`, ...). We ask Kevin for no rename. We do not use the spec's **host system** (§1).
- An endpoint is a **universal inference processor**. Image, speech, video and multimodal ensembles become ordinary expressions on the same engine, telemetry and cost accounting (§8).
- **Most of this is now in Part A §1.4:** `Mount`, `Evaluator`, `Dry run`, `Degradation`, the response ladder, `Scheme adapter`, `Flow constraints`, `Attribution`, `Run handle` and the `application/url4-envelope+json` switch. What is still open is in the companion PDF.

1. Terms

Eleven words, one meaning each. Nearly all are now the spec's own; Appendix D has the full map. This is the short list for §2–§9.

OUR WORD	MEANING	IN THE SPEC (PART A §1.4 @ 2A939BFF)	IN THE SDK	IN THE ENGINE
Endpoint	An interface on a node that evaluates url4. Reached at an endpoint path.	§1.4.2, same word. Now an interface, separate from the path that names it.	an entry in <code>Url4Node</code> 's endpoint registry	a route such as <code>/anthropic/<model></code>
Endpoint path	The URI path (<code>/claude</code>) that names an endpoint, relative to the node address.	§1.4.2, new .	the registry key	the route
Node	An origin that serves endpoints. <code>/</code> is the default. Answers discovery.	§1.4.2, same word.	<code>Url4Node</code>	the App, after this change
Node address	The node's network address: URI scheme plus RFC 3986 authority.	§1.4.2, new .	<code>Url4Node</code> 's origin	the App's origin
Target	The endpoint URI an expression is sent to: node address + endpoint path.	§1.4.3, sharpened. Was "an absolute/relative URI defining the Node + Endpoint".	<code>FetchRequest</code> 's destination	the route the Runner calls
Mount	How an endpoint path binds to its evaluator: <code>local</code> (in-process), <code>command</code> (subprocess), <code>proxy</code> (declared target).	§1.4.3, taken from this document word for word . Was not defined.	<code>endpoint()</code> , <code>[commands]</code> , <code>—</code>	in-process handlers only
Evaluator	What runs an expression: resolves sources, fans out, reduces, runs the intent.	§1.4.3, adopted and widened: it may or may not also be the intent processor.	<code>url4.dag.run</code>	<code>Url4Executor</code> in the Runner
Intent processor	What turns resolved context plus intent into a result: a model call, a script, a command.	§1.4.3, same word.	endpoint handler	one <code>httpx</code> call to aigateway
Requestor	Whoever sends an expression to a target.	§1.4.3, same word.	<code>Client</code>	product client
Request tree	The nodes and endpoints one expression touches.	§1.4.3, adopted with a change . The spec now says an evaluator may expand the tree. It drops "strict tree, never a graph" (Part H §29.1). Delta 14 asks if that is on purpose.	the compiled DAG, per endpoint	one run
Capabilities document	JSON a node publishes: its endpoints, mounts, features, delivery modes and schemes.	§1.4.5 and Part G §27.1–§27.2. Now also names the <code>Capabilities</code> response header.	none	none

Words we do not use in the protocol: ensembler, orchestrator, swarm, composition, fusion. "Fusion" stays product copy. Plan is gone too: the spec now defines **dry run** (§1.4.3), and §10 asks only what shape it takes.

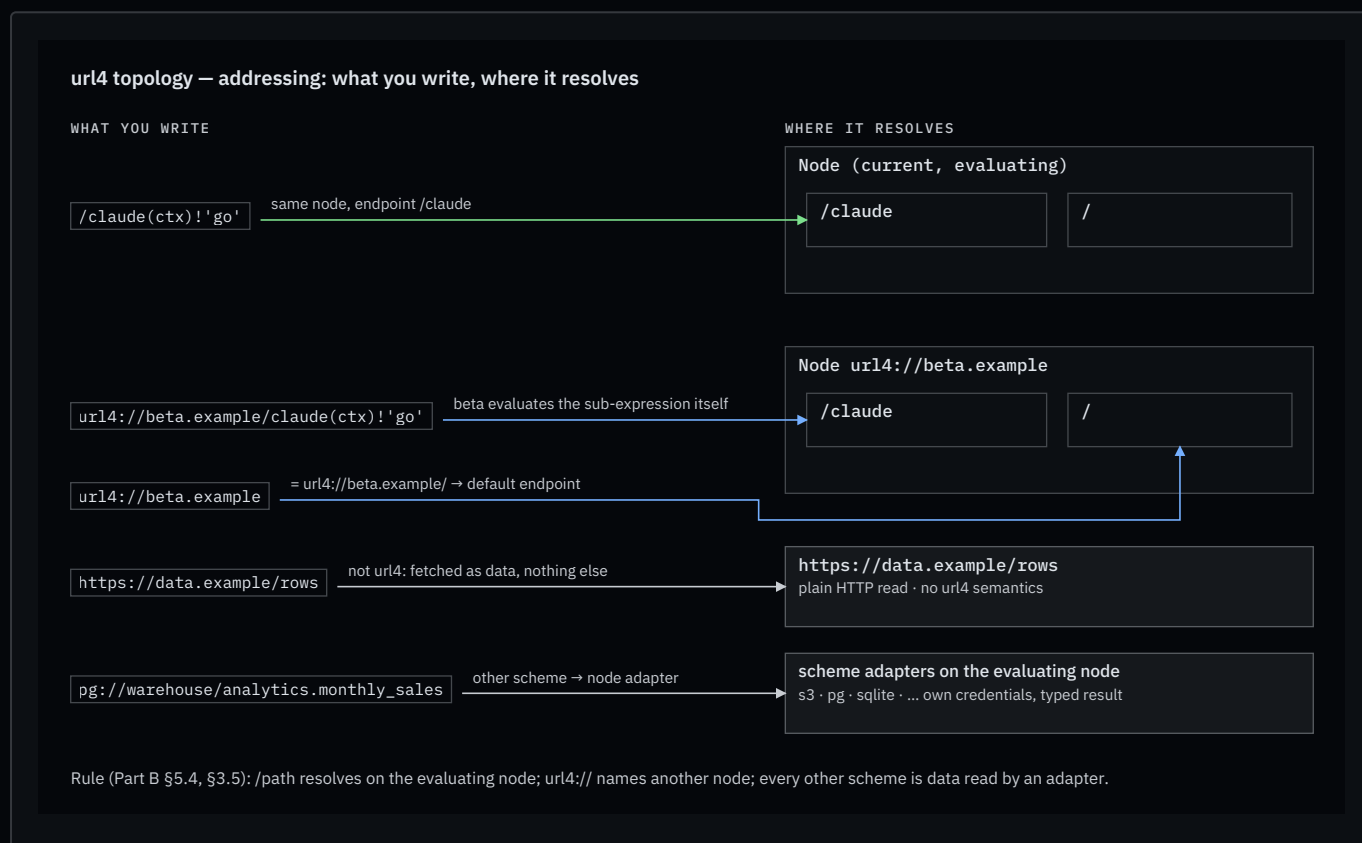
"Host" is not one of our words. Part A §1.4.2 defines a **host system**: a physical or virtual machine that runs one or more nodes. That is a fine deployment word. It changes nothing here. But §1.4.4 and the §1.4.3 **Scheme adapter** row then use "host" to mean the node. It is the node that negotiates delivery, picks a rung on the response ladder, and holds scheme credentials. A host system does none of that. This document never writes "host". Delta 13 asks Kevin to fix those four rows.

Naming note: Parts C–I on the **url4-refactor** branch still use **abc://** and **ABC-***. Parts A/B use **url4://**. This document writes **url4** and **URL4-*** throughout. That assumes the rename Kevin lists as open question 19 (Part I §43).

2. Addressing

Three address forms, plus any other scheme as data. Part B §3.1.1, §3.5 and §5.4 fix their meaning. We add one rule about the root.

Part A §1.4.2–§1.4.3 names the parts. A **node address** is scheme plus RFC 3986 authority. An **endpoint path** names an endpoint relative to it. A **target** is the two together: the endpoint URI an expression is sent to. The forms below are targets.



- `/claude(ctx)!'go'` — an endpoint on the node doing the evaluating. Resolves to `url4://<current node>/claude` (Part B §5.4).
- `url4://beta.example/claude(ctx)!'go'` — an endpoint on another node. That node evaluates the sub-expression itself (Part B §3.1.1; SDK invariant OME-535).
- `url4://beta.example` — the node's default endpoint. **Our rule:** `url4://node` $\equiv$ `url4://node/`.
- `https://data.example/rows` — plain HTTP data. No url4 headers, no envelope. `@` is an error (Part B §3.5).

Root and version. `/` is the node's default endpoint. The spec makes `/v1` the version path (Part D §19.1). But its own examples write `abc://thepost?q=...` with no path (Part I §41.27). We keep both: `/` is the current version, `/v1` an alias. Two things to settle with Kevin. Part D §19.4 says the `v` parameter beats the path. §19.3 says a nested expression takes its version from the path it targets — and a bare node address has no path. The SDK's `url4 serve (/v1)` and `Client` (default `/v1`) move to `/` (Appendix B).

Scheme inheritance. A bare relative data URI takes the scheme of the request that carried it (Part B §5.4.1). Under `url4://` it is a url4 read. Under `https://` it is a plain read.

Any scheme is a source. Now a spec rule. Part A §1.4.3 `Scheme adapter` took this from us: a non-url4 reader mounted on a node, typing its result, listed in capabilities. So the rules below describe the spec. (Its row says "host's" credentials; it means the node's — delta 13.) The SDK already has the slot: `FetchRequest.kind` is `url4`, `http`, `relative`, or `other`.

```
(sales=pg://warehouse/analytics.monthly_sales,
 logo=s3://brand-assets/logo.png;accept=image/png,
 notes=sqlite:///srv/app/notes.db/notes,
 policy=https://data.example/policy.md)!'Draft the Q3 update. Use the logo.'
```

Rules:

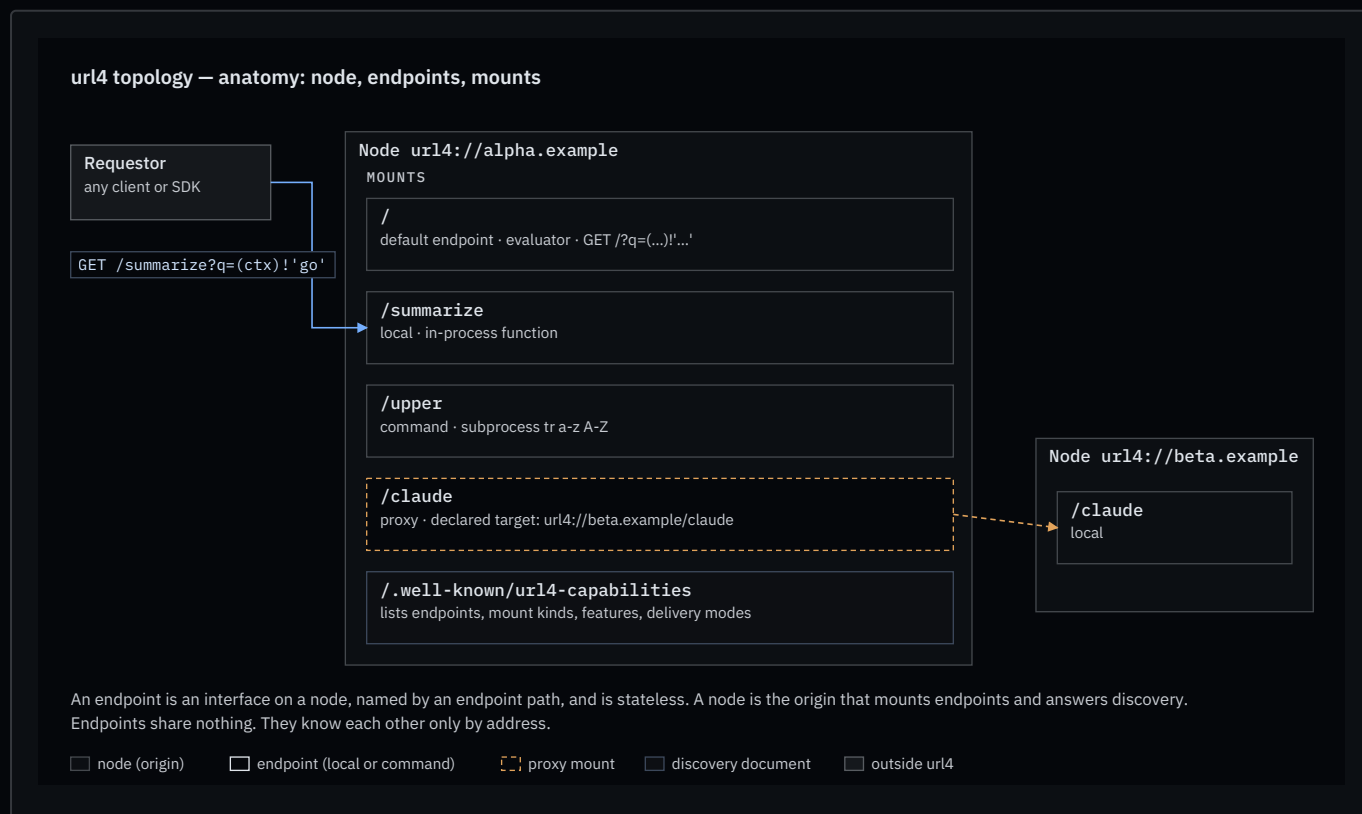
- **The evaluating node resolves it, with its own credentials.** `pg://warehouse` names a connection the node knows. The expression never carries a secret, because the expression is the shareable audit record (Part A §1.2). The spec's delegated token can only target `abc_node` or `http_source` (Part H §31.4). So node-owned credentials are the only way a non-HTTP scheme gets any.
- **The adapter types the result** (§8). An S3 object keeps its stored `Content-Type`. A table or query returns `application/json` rows, or `text/csv` when asked with `;accept`. Rows are a collection, so `pg://warehouse/analytics.monthly_sales*(row)!'summarise $item'` iterates them (Part B §5.3).
- **Advertised, or refused.** The node lists its schemes in the capabilities document. An unlisted scheme is `unsupported_mode`, permanent (Part B §3.5). Access control and consent apply as for any source (Part B §5.6.4). The source is not url4-aware: the envelope marks it `abc_aware: false` and attribution stops there (Part G §26.4), as for any plain HTTP read.
- **Hexagonal.** A scheme adapter is an `IOLayer` adapter registered by scheme. The core never imports it. `url4 serve` gets a `[schemes]` table beside `[data]`.

What a scheme's path means (bucket and key; database, schema and table; file and table) is the adapter's contract, not the grammar's. The grammar sees only `scheme://authority/path`.

3. Endpoint contract

An endpoint is a function. That is the whole idea.

Part A §1.4.2 says it more precisely: an endpoint is an interface on a node, named by an **endpoint path**. Two words. "Endpoint" is the function. "Endpoint path" is `/claude`.



An endpoint:

- **MUST** answer `GET <path>?q=<expression>` and evaluate the expression. Every endpoint evaluates url4. It may walk a DAG before it reaches its own intent processor. There is one grade of endpoint, not two.
- **MUST** be stateless per call. It keeps nothing between calls. Its own data sits behind `@` (Part B §5.6), not inside it. The spec's one exception is an agent session (Part G §28). The coordinating node holds the session and transcript for the session's life. Participant endpoints stay stateless, because the transcript travels with every turn. `;coord=` picks the mode (`session`, `debate`, `roundrobin`, `freeform`). The endpoint issues the session id (`Session-Id` header); the requestor does not.
- **MUST** return the envelope (Part D §17) and say which delivery mode it used (Part C §11.4).
- **MUST NOT** resolve `@` inside a sub-expression sent to another node (Part B §5.6.3.1).
- **MUST** answer sync. **SHOULD** offer stream (SSE), WebSocket and async, and say which (§5, §6). Asked for more than it has, it answers with the richest mode it does have. That is never an error.
- **MAY** be mounted on any node, or be its own origin. A Lambda-style function with its own URL is a node with one endpoint.
- Speaks GET. A WebSocket, when offered, is the same GET with `Upgrade: websocket` (§6). Other verbs are 405, except two this document adds: `DELETE` on a run handle (§6) and, if adopted, `OPTIONS` (§5).

Endpoints share nothing. An endpoint knows another only by address, and only because the expression named it.

4. Node contract

A node is an origin. It routes. It does not evaluate.

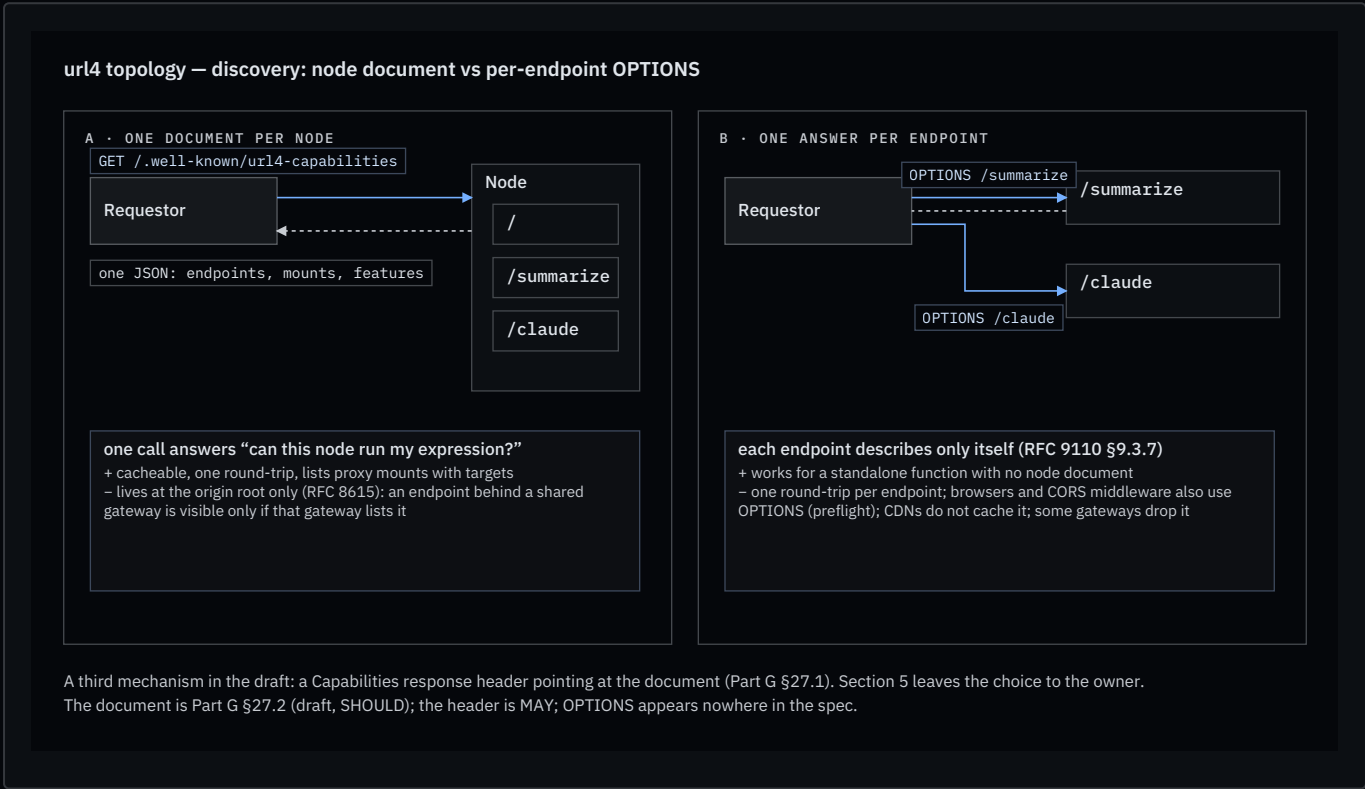
- Mounts one or more endpoints at paths. `/` is the default.

- Mount kinds: `local` (a function in the process), `command` (a subprocess), `proxy` (forwards to a declared target on another node). Now Part A §1.4.3 `Mount`, beside the spec's processor types `internal` / `function`, `code` and `abc_delegate` (Part G §27.3); one fix to send back is delta 15. A proxied relative call is the spec's delegation: the caller has resolved the context, so the wire carries materialised sources plus the intent, never a raw sub-expression. A `command` mount runs code, so Part H §36's pinning and signing rules apply.
- Lists proxy mounts and their targets in its capabilities document. Attribution and consumer disclosure (Part H §29.2.1) need the real target, so proxies are never hidden. A source may cap redistribution depth or name allowed consumers (Part H §29.2.2). A proxy target is a consumer.
- Serves discovery (§5) and the version alias `/v1` (§2).
- A local SDK process that mounts remote endpoints under local names is a node. That is how "run the ensemble on my laptop, but call `/claude` as if it were mine" works.

5. Discovery — the open decision

Three ways for a requestor to learn what a node can do. Draft Part G defines the first two: a capabilities document (SHOULD) and a `Capabilities` response header that MAY point to it (Part G §27.1). OPTIONS is ours. It is not in the spec. The spec also uses `.well-known` for the policy registry (`/ .well-known/abc-policy`, Part H §30.2), so two well-known documents will exist side by side.

Settled since 2026-09-08. Part A §1.4.5 writes `/ .well-known/url4-capabilities` (Part G still says `abc-capabilities`) and names the `Capabilities` header, C below. The choice is still the owner's. We still recommend A + C.



	A · NODE DOCUMENT GET <code>/ .WELL-KNOWN/URL4-CAPABILITIES</code>	B · PER ENDPOINT OPTIONS <code>/<ENDPOINT></code>	C · CAPABILITIES RESPONSE HEADER
"Can this node run my expression?"	one call, whole answer	one call per endpoint in the expression	pointer only; still one fetch of A

	A · NODE DOCUMENT GET /.WELL-KNOWN/URL4-CAPABILITIES	B · PER ENDPOINT OPTIONS /<ENDPOINT>	C · CAPABILITIES RESPONSE HEADER
Standalone function with its own origin	works: the document sits at that origin's root	works	works
Endpoint behind a shared, non-url4 gateway	invisible unless the gateway lists it (RFC 8615: root of the origin only)	works: the endpoint answers for itself	works: any response can carry the pointer
Caching	ordinary GET: ETag , CDN, browser cache	not cached by CDNs or browsers	rides on the cached response
Browsers and CORS	plain GET	preflight uses the same verb; CORS middleware often answers first (RFC 9110 §9.3.7 allows a body, browsers ignore it)	plain header
Gateways and serverless platforms	fine	some strip or auto-answer OPTIONS	fine
Proxy mount targets (attribution)	listed in one place	per endpoint	via A
Spec status	Part G §27.1, SHOULD; schema Part G §27.2	not in the spec	Part G §27.1, MAY

One hazard for all three: the Engine's JWT header is called `URL4-Capability`. The document is "capabilities". Appendix B proposes renaming the header.

Capabilities document: the spec's shape, plus our additions. Part G §27.2 defines the top level: `abc_version`, `node` (one canonical URI: our node), `self_ref_support`, `collections[]` (`id`, `path`, `description`, `element_schema`, `formats`, ...), `behaviors`, `intent_processors[]` (`id`, `type` ∈ `internal` | `abc_delegate` | `http_delegate` | `code` | `function`, `endpoint`, `capabilities`), `processor_policy`, `mode_support`, `agent_profiles`. Content types are `consumes` / `produces` from Part F §25.4. Sub-paths are collections (Part G §27.4). Our additions are marked. They are what makes a processor reachable as an endpoint:

```
{
  "abc_version": 1,
  "node": "url4://alpha.example",
  "default_processor": "summarize", // ours
  "intent_processors": [
    { "id": "summarize", "type": "internal", "path": "/summarize", // path: ours
      "consumes": ["text/plain"], "produces": ["text/plain"],
      "delivery": ["sync", "stream"] }, // delivery: ours
    { "id": "upper", "type": "code", "path": "/upper" },
    { "id": "claude", "type": "abc_delegate", "path": "/claude",
      "endpoint": "url4://beta.example/claude" }
  ],
  "schemes": ["s3", "pg", "sqlite"], // ours (§2)
  "collections": [ { "id": "science", "path": "/science", "self_ref_eligible": true } ],
  "self_ref_support": true
}
```

One point for Kevin. The spec's document describes one node. Its processors are addressed by `id`, and its sub-paths are collections. Our endpoints need a path each. `intent_processors[].path` is the smallest change that gives both.

Owner decision (open). Pick one:

- ☐ **A only** — node document is a MUST; nothing per endpoint.
- ☐ **B only** — OPTIONS per endpoint is a MUST; no node document.
- ☐ **A MUST + C SHOULD** — the spec's pair, one level stronger than its SHOULD/MAY.
- ☐ **A MUST + B SHOULD** — the document indexes the node; an endpoint may also answer OPTIONS with its own entry.

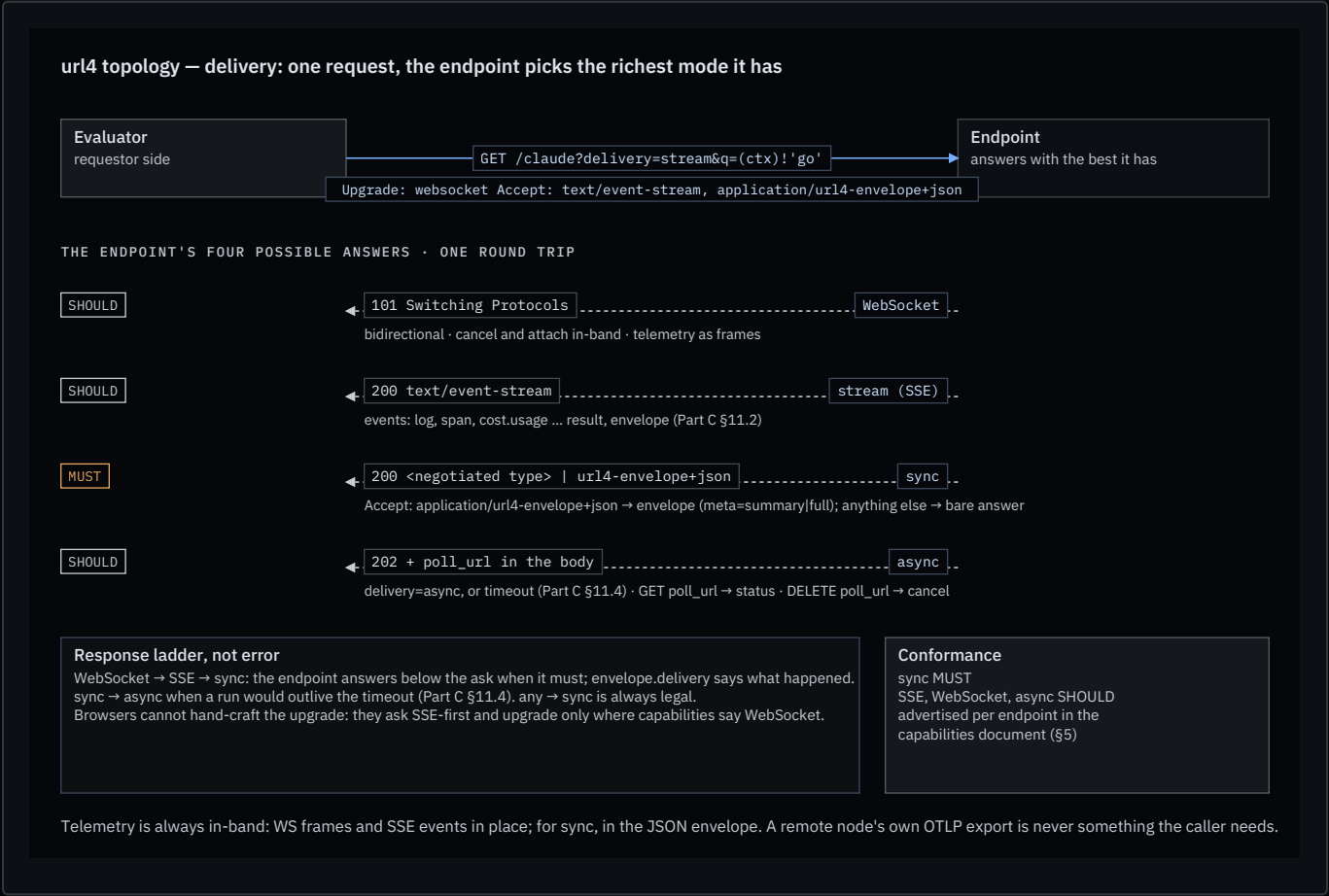
Recommendation, after reading Part G: **A MUST + C SHOULD**, B dropped. The `Capabilities` header solves the gateway case that OPTIONS was for, and it is already in the draft.

6. Delivery and transport

This section is now in the spec. Part A §1.4.4 has four `delivery` values: `sync` (one body), `stream` (SSE on the same GET), `async` (202 plus `poll_url`) and `websocket` (101 on the same GET). Only `sync` is a MUST. The **response ladder** — answer with the richest mode you have, WebSocket → SSE → sync; answering lower is a reported degradation, not an error — and **degradation** are spec terms too.

Part C §11 has not caught up: three modes, two ladder edges. Kevin's row says SPEC SECTION NEEDS UPDATING (delta 4). His rows also say "host" for node (delta 13).

Part I §43 question 26 puts `ws://` sources out of scope. That is a different question from a WebSocket answer on the endpoint's own GET. Adding `websocket` as a delivery value settles it.



One request, the endpoint picks. The evaluator sends the richest ask it can, once:

```
GET /claude?delivery=stream&q=(ctx)!'go'
Upgrade: websocket
Accept: text/event-stream, application/json
```

The endpoint answers with the best it has. RFC 6455 lets a WebSocket handshake ride an ordinary GET. RFC 9110 lets a server ignore Upgrade and answer normally. So one round trip covers the whole response ladder:

ENDPOINT ANSWERS	MEANING	TELEMETRY TRAVELS AS
101 Switching Protocols	WebSocket: two-way; cancel and attach in-band	frames, same names as the SSE events
200 text/event-stream	stream: SSE on the same GET (Part C §11.2)	SSE events, then result, then envelope
200 text/plain or application/json	sync: bare answer, or the envelope, by Accept	envelope meta (below)
202 + poll_url	async: delivery=async ; the 202 body carries job_id , rid , poll_url (Part C §12.5); GET it for status, DELETE it to cancel	on the handle

Rules:

- sync is the only MUST. SSE, WebSocket and async are SHOULD. Each endpoint says which it offers in the capabilities document (§5). A serverless function with only sync and SSE conforms.

- **Response ladder, not error.** WebSocket → SSE → sync. Answering below the ask is normal. The envelope's `delivery` field says what happened (Part C §11.4). The degradation is reported like any other (Part C §10.2.2). `any → sync` is always legal.
- **Async.** Ask for it with `delivery=async` (Part C §9.1). The endpoint may also switch to it when a sync run would outlive the timeout (Part C §11.4). The Engine's `Prefer: respond-async` becomes an alias. The run handle is `poll_url` in the 202 body. A `Location` header that mirrors it is our addition.
- **Browsers go SSE-first.** A browser cannot set `Accept` on a WebSocket handshake. So a browser evaluator asks for SSE, and upgrades only where the capabilities document offers WebSocket.
- **The Engine's product session** keeps its WebSocket at `/ws`. It is one case of the WebSocket rung, not a separate protocol.
- **Telemetry is always in-band.** In every mode the caller gets logs, spans and cost on the same connection or in the envelope. A remote node's own OTLP export is its business. The caller never depends on it (§7).

Telemetry per mode. WebSocket and SSE carry it in place. An SSE body is a sequence of named events (`event: log`, `event: span`, `event: cost.usage`, then `event: result`, `event: envelope`), in order, on one response. The spec already works this way: Part C §12.5 is an SSE event catalog. Our three signals are three more names. SSE lacks only the return path: no in-band cancel or attach.

Sync has no room for that. The body is the answer. The spec always returns the JSON envelope and uses `Accept` for the result's content type (Part C §9.1, Part F §25.2). We add one switch on top, named so it cannot collide:

KNOB	GOVERNS	VALUES
<code>Accept</code> header	the wrapper, by one dedicated type	<code>application/url4-envelope+json</code> : the envelope (Part D §17). Anything else: the bare answer in its content type, no logs, no telemetry — what <code>url4 serve</code> returns today. Now Part A §1.4.4 <code>Envelope</code> .
<code>meta</code> param	how much the envelope carries	<code>none</code> : result and status. <code>summary</code> : counts, latency, cost. <code>full</code> : per-source detail, nested child envelopes (Part D §17.3), and a <code>telemetry</code> block with logs and spans. The <code>telemetry</code> block is ours. Per the Hybrid Rule it is present and <code>null</code> at <code>summary</code> (Part D §17.2.2).
<code>fmt</code> param	the shape of the answer content	<code>text</code> , <code>markdown</code> , <code>json</code> (Part C §9.1). Orthogonal: a JSON envelope can wrap a markdown answer.

An envelope answer with no `meta` gets `summary`. The spec's default is `none` (Part C §9.1). So asking for the envelope always buys the cheap view; `meta=full` is the explicit debug switch. What an endpoint exposes is its choice (Part D §18.4). A production endpoint may answer `meta=full` with the `telemetry` block redacted. It MUST keep the structure and use `null` or `"redacted"`. It must never drop fields silently. A development endpoint hands over everything. Same envelope, same evaluator code, different policy.

Cancel. The spec calls cancellation a gap and offers two shapes: `DELETE <poll_url>` or a `cancel=<rid>` parameter (Part C §16.2). We take the first. Over WebSocket, cancel is in-band. Otherwise `DELETE` on the `poll_url`, which the Engine already does. New terminal state `cancelled`. SSE event `request.cancelled`. Partial result returned when quorum was already met, as §16.2 asks. Note for Kevin: `status: "cancelled"` is not yet in the status enum of Part D §17.4.

7. Telemetry

Three signals, one trace, as in the doctrine skill. What changes is where a one-shot GET puts them.

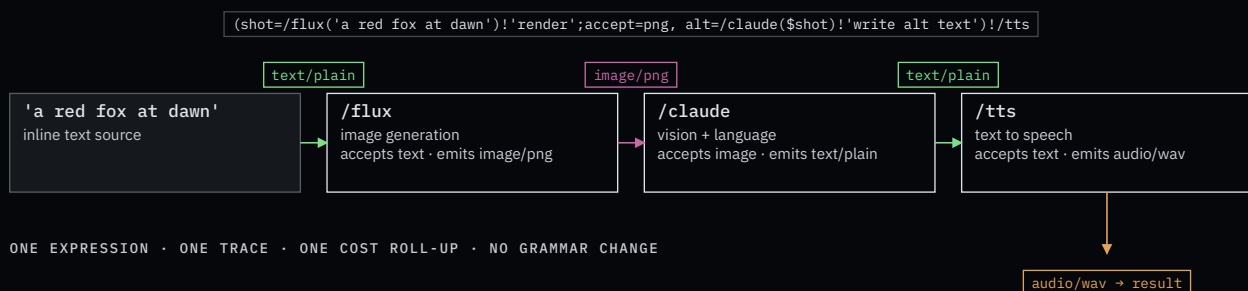
- **sync:** in the body, only when the caller asked for the envelope (`Accept: application/url4-envelope+json`, §6). `meta` sets the level: `summary` by default, `full` adds logs and spans (Part D §17.2). At `meta=full` a child's envelope nests in `source.envelope` (Part D §17.3). Each endpoint aggregates only what it saw itself (Part D §18.1). A bare `text/plain` answer carries nothing.
- **stream and WebSocket:** as events. Logs, spans, `cost.usage` (self and subtree), then `result`, then `envelope`. Same names in both.
- **durable:** OTLP export to the trace backend. There is no separate "Enclave" store. The exporter replaced it.
- **What the spec already carries:** 21 SSE event types (Part C §12.5), `meta.total_cost` and `budgets_spent` in the envelope (Part D §17.1.2.8, Part E §24), and an endpoint-local audit log (Part H §34). Our logs, spans and `cost.usage` events add to that catalog. They replace nothing. Cost stays a separate event; `total_cost` is its roll-up. Every streamed event with content already MUST carry `content_type` (Part C §12.5). §8 builds on that.

This closes doctrine item F4. **Idempotency:** a url4 GET is idempotent in the HTTP sense (RFC 9110 §9.2.2). Repeating it has no extra server-side effect. Results need not be identical. Part C §15.1 mixes the two up (Appendix A). The spec lists one real exception: a repeat that would exceed a spent budget fails with `budget_exceeded` (Part C §15.2).

8. Typed payloads: an endpoint is a universal inference processor

We have treated an endpoint as text in, text out. That is a habit, not a spec rule. The spec says a source is "a URI, text, or nested expression" and an intent processor is whatever executes the intent for the endpoint (Part A §1.4). Nothing says the payload is a string.

url4 topology — typed payloads: every edge carries a media type



How a binary result travels

small: inline — `result.content` as a data URI + `result.content_type` (Part G §26.2.3), or the bare response body with its Content-Type
 large: by reference — `result.artifact`, an `https://` data URL
 the next endpoint fetches like any source (Part B §3.5)
 threshold: the endpoint's, advertised as `inline_max_bytes`

How an endpoint declares types

source: fetched Content-Type · text/plain · emitting endpoint's type
`accept=<alias>` says what the consumer wants (Part B §4.2, Part F §25.5)
`ct_mismatch` says what to do when it differs (Part B §8.1.2)
 capabilities: consumes: [...], produces: [...] per processor (Part F §25.4)
 an evaluator can type-check the expression before spending

Text stays the default. Image generation, speech, video and multimodal ensembles become ordinary url4 expressions on the same engine.
 Over WebSocket a binary frame carries bytes; over SSE a large payload always goes by reference.

Proposal. Every edge in an expression carries a **typed payload**, the way a ComfyUI edge carries an image, a latent or a mask. The type system is the media type: `text/plain`, `image/png`, `audio/wav`, `video/mp4`, `application/json` for embeddings and structured data. Text stays the default. No grammar change.

```
(shot=/flux('a red fox at dawn')!'render';accept=png,
alt=/claude($shot)!'write alt text')!/tts
```

Edges: text → image → text → audio. Three endpoints, three processors, one expression, one trace, one cost roll-up.

What the spec already gives us

- `;accept` is a source-level execution annotation (Part B §4.2). It takes a **short alias** (`json`, `csv`, `markdown`, ...), never a MIME type, because `/` would read as a relative URI (Part F §25.5). The registry has no image, audio or video rows yet. `png`, `jpeg`, `wav`, `mp4` are proposed in Appendix A. `ct_mismatch` has four values, `fail` | `ignore` | `transform_ignore` | `transform_fail`, default `ignore`. It applies only when `;accept` was set (Part G §26.3). The spec puts input negotiation out of scope. An unusable format is an intent-execution failure, not a source failure (Part F §25.10).
- The spec already has a weighted `image/png` source feeding a text intent (Part I §41.20). Part G §26.2.3 already says how binary content travels in LLM mode: a data URI, `data:<mediatype>;base64,...`, with raw bytes for RDS intents. The SDK carries a media type on every fetch (`FetchRequest.media_type`) and parses collections by declared type (Part B §5.3.7). The Engine already splits results by size: inline, spilled to a content-addressed artifact, or refused above a hard cap.

Three rules the spec does not yet give (Part F, §25)

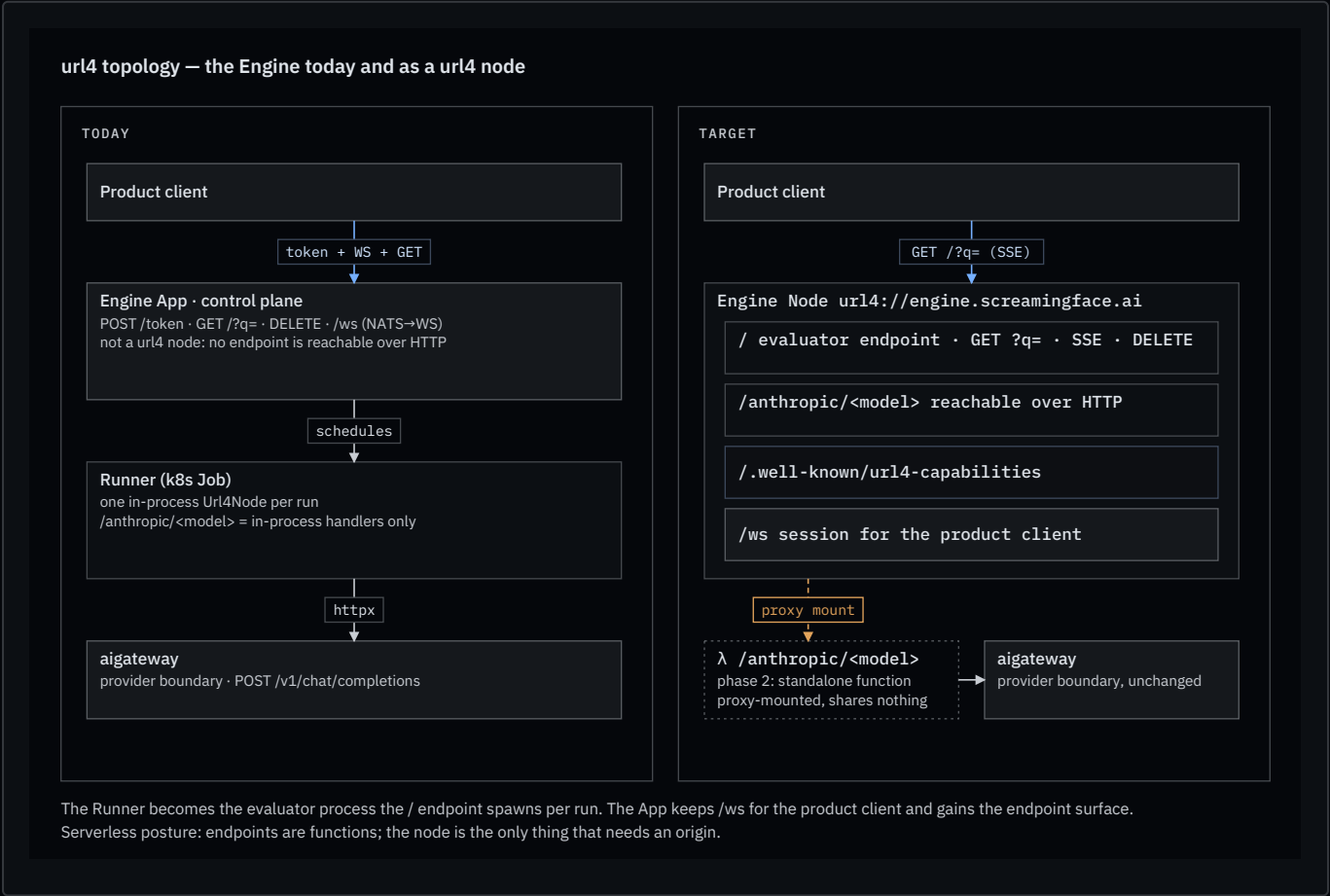
1. **A source declares its type.** A URI source has the `Content-Type` it was fetched with. Without one, the endpoint infers from the path or sniffs, and falls back to `application/octet-stream` (Part G §26.2). Inline text is `text/plain`. A nested expression has the `result.content_type` its endpoint emitted (Part D §17.1.2.1). `;accept` says what the consumer wants. `ct_mismatch` says what to do when the two differ.
2. **A binary result travels inline when small, by reference when large.** Bare sync: the response body is the bytes, with its `Content-Type` (Part F §25.5 allows raw binary outside the envelope). Envelope: `result.content` carries a small payload as a data URI with `result.content_type` set — the spec's own form. A large payload becomes `result.artifact`, an `https://` URL the next endpoint fetches as plain data (Part B §3.5). The inline limit is the endpoint's, stated in its capabilities. Over WebSocket a binary frame carries the bytes. Over SSE a large payload always goes by reference. Two cautions from the drafts. This is not truncation, which Part G §26.2.1 forbids. And an artifact URL is a redistribution, so it inherits the source's flow constraints and depth cap (Part H §29.2.2).
3. **A processor says what it accepts and emits.** The spec's names are `consumes` and `produces` (Part F §25.4), per processor in the capabilities document. An evaluator can then type-check an expression before it spends anything. The dry-run question in §10 builds on this.

```
{ "id": "flux", "type": "internal", "path": "/flux",
  "consumes": ["text/plain"], "produces": ["image/png"], "inline_max_bytes": 262144 },
{ "id": "claude", "type": "internal", "path": "/claude",
  "consumes": ["text/plain", "image/png", "image/jpeg"], "produces": ["text/plain"] },
{ "id": "tts", "type": "internal", "path": "/tts",
  "consumes": ["text/plain"], "produces": ["audio/wav"] }
```

Why it matters. Image generation, text-to-speech, speech-to-text, video evaluation and multimodal ensembles all become ordinary url4 expressions on the same engine. Same telemetry, same cost accounting, same attribution. aigateway stays the provider boundary; it already fronts image and audio providers. The Engine's artifact store is rule 2 as built.

Open. A media type for embeddings (`application/json` with a profile, or a vendor type). How attribution weights and `tokens=` budgets apply to non-text sources (Part E is silent; there is no `bytes=` or `seconds=` budget key). Settled by the draft: `fmt` stays a structure hint. A future version MAY retire it in favour of `Accept` (Part F §25.8).

9. The Engine as a node



	TODAY	TARGET
App	control plane: POST /token , GET /?q= , DELETE , /ws bridged from NATS	a node : / evaluator endpoint (GET ?q= with WebSocket, SSE, sync and async answers, DELETE), model endpoints over HTTP, capabilities, /ws kept for the product client
Runner	k8s Job with one in-process Url4Node ; model routes are in-process handlers	the evaluator process the / endpoint spawns per run
Model endpoints	/anthropic/<model> reachable only inside the Runner	reachable over HTTP on the node; later each one a standalone function, proxy-mounted
aigateway	provider boundary	unchanged
Endpoint-to-endpoint	unused SDK code	the normal path

Phases. (1) Node surface on the App: discovery per §5, GET /?q= answering SSE and sync (WebSocket via Upgrade where the App already has it), DELETE . (2) Model endpoints over HTTP. (3) Standalone model functions, proxy-mounted, listed in the capabilities document.

Serverless posture. An endpoint is a function. It needs no origin of its own. Only the node needs one. The Runner is already function-shaped. The App is not: it holds an audience count in memory, runs two perpetual tasks, and gates runs on an attached WebSocket. Those belong to the product session, not the endpoint surface. They stay in the App.

Appendix C — where the spec lives

- Parts A and B, v0.5 DRAFT (2026-07-10): `secondbrain/kevin-mcdonough/docs/adrs/URL4-Spec-A.md` , `URL4-Spec-B.md` .
- **Part A §1.4 as cited here** is not that file. It is the version in `OpenMined/screamingface-design PR #19 (url4/terminology-refresh` , commit `2a939bff` , opened 2026-09-15). That PR is a **draft against** `main` with no review. Every "`§1.4.x`" anchor in this document resolves there and nowhere else yet.
- Parts C–I: draft markdown on the `url4-refactor` **branch** of `OpenMined/screamingface-design` , `kevin-mcdonough/docs/adrs/refactor/URL4-Spec-{C...I}.md` . Cut 2026-04-28 from the v0.4 text. Not yet reviewed. Never merged to `main` , where the site shows "Not yet written" stubs. Cited here as "Part X §N". The same branch holds the v0.4 monolith (commit `f28608a`). The v0.2 monolith this document was first written from is `8a052dc` ; its numbering diverges from §21 on.
- Public docs: `public-docs/src/pages/learn/Url4Page.vue` . They use "fusion" and "typed DAG". The spec uses neither.
- Doctrine: `.claude/skills/url4-engine/SKILL.md` , updated with this document.